

Linking for Another Crate: Cross-Unit Native-Library Shadowing in Cargo's Build System

ANONYMOUS AUTHOR(S)

Cargo build scripts influence compilation by printing directives that Cargo attributes to the emitting package. Most directives stay local, but the native-library directives do not: `rustc-link-search` and `rustc-link-lib` propagate to the final link of the top-level binary. We show this creates *cross-unit native-link authority*: one crate can add a search path that resolves another crate's library request. On released Cargo, a distance-2 transitive crate that declares no links key and emits one ordinary `rustc-link-search` line makes the final binary link and execute attacker native code. The witness is not synthetic: an end-to-end build against the unmodified published crate `libsqlite3-sys 0.38.2` links the legitimate SQLite archive in the baseline and the attacker's under the shadow. The attack survives build-script confinement, under Cackle with Bubblewrap the script only emits a permitted directive while the harm occurs later at link resolution, and is hard for per-crate review to see, because neither crate alone contains the cross-crate resolution. The exposure is not a forced configuration: in their *default* build, popular `-sys` crates link *system* libraries by search-resolved requests on both Windows (`curl-sys`, `libgit2-sys`, `libssh2-sys` → `crypt32/secur32/bcrypt`) and Linux (7 of 11 crates → `ssl/crypto/z/...`), and we run attacker code in the shipped binary of a real, default-features `curl` build.

We map the boundary (`rustc-link-arg`, `rustc-cfg`, and indirect `DEP_*` do not cross the same way; links uniqueness gates metadata, not search paths; `static=` resisted in our tests) and measure prevalence over a uniform crates.io sample and 185 resolved dependency graphs, of which 39.5% (95% Wilson 32.7-46.6%) already contain the victim precondition (a crate requesting a search-resolved native library); we report this as a candidate rate, not an exploitability rate, and observe no naturally occurring *unrelated* cross-crate collision (the two found are legitimate provider pairs). We frame the shadow as a violation of *resolution integrity*, every build-time resolution decision should be attributable to the requesting unit's own provenance, and characterize when a directive carries cross-unit authority (its inputs are merged across units *and* the resolving step is unscoped, so the boundary above is a corollary), with provenance-scoped resolution the *minimal* sound enforcement *among provenance-based path filters*. It resolves each crate's request only within its own search paths, the system defaults, and declared provider dependencies. We evaluate the ideal fail-closed resolver (modelled and machine-checked in Lean 4) and enforce it on real builds: a strict linker wrapper passes all 5/5 adversarial conditions with no attacker selection, while a separate in-tree Cargo patch is a partial directory-level variant. Strict enforcement builds all 45/45 tested graphs at zero annotations, a one-line consumer declaration migrates the one rejected legitimate class, and 27/27 real native-linking crates build unchanged; a build system is exposed iff it has a resolution step that is both merged and unscoped, which places Cargo and CMake's `link_directories` on the exposed side and target-scoped CMake, hermetic Bazel/Nix, and node-gyp on the safe side. We disclosed to the Rust security team, who place it outside their security-response scope (dependency trust) and see no privilege escalation, which we accept: the contribution is not a new capability but resolution integrity and a link-step defense-in-depth mechanism, complementing the best-effort build-script sandboxing Rust already plans but which cannot reach the link-resolution step.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → **Software evolution**.

Additional Key Words and Phrases: software supply chain, build scripts, dependency review, Rust, Cargo

1 Introduction

A Cargo build script steers compilation by printing directives Cargo attributes to the declaring package [27]. Most act on that package alone (`rustc-cfg`, `rustc-env`), but the native-library family is different: `rustc-link-search` and `rustc-link-lib` *propagate to the link of the top-level binary*, the mechanism by which a `-sys` crate makes the final artifact link a C library. We show

this documented convenience is also an unexamined security boundary: it gives a crate *cross-unit authority*, power over a compilation unit it is not part of, reaching the shipped binary.

Concretely, on cargo 1.98.0, a crate at distance 2 that declares no links key can emit a single ordinary-looking rustc-link-search line pointing at a directory it ships, causing the final linker to resolve a *different*, legitimate crate's rustc-link-lib to the attacker's library, so attacker native code links into and executes in the shipped binary. Its build script does nothing else (no network, filesystem write, or spawned process), so the attack *survives* the build-script confinement being built to contain build-time code execution, emitting a directive is something a sandbox must permit, and it is unattributed: no per-crate review reveals that one crate's search path resolves another's rustc-link-lib, because the malice is in the *composition* of a per-crate directive with Cargo's global, unscoped resolution, not in any crate's reviewed code.

Build-script security research has looked elsewhere. The August 2026 arrayref compromise [33] (a script that downloaded and ran a payload) is build-time *code execution*, and its proposed defenses inspect code or restrict capabilities of named crates: Cargo Vet and cargo-crev share dependency audits [2, 19]; Cackle checks API permissions and can sandbox scripts [15]. Library shadowing is known at other layers, a runtime loader problem (LD_PRELOAD) [18, 23], and a build-time footgun some build systems guard against (CMake's per-target scoping) [12]. What has *not* been recognized is the same confusion as a cross-crate supply-chain attack: a transitive, unsigned crate shadowing another crate's library across compilation units, where per-crate review and confinement are exactly the defenses that miss it.

We are precise about what we do and do not claim. We claim no privilege beyond the code execution a build script already has; the propagation is documented behavior, not a Cargo defect. The finding is a blind spot in the *defenses* being built to relax dependency trust, build-script sandboxing and per-crate review, neither of which reaches the link-resolution step this composition abuses, closed by a resolution-scoping mechanism those defenses lack.

The exposure is not contrived: in their *default* build the HTTP, git, and SSH stacks many projects depend on link the Windows cryptographic system libraries through ordinary search-resolved directives a single unsigned transitive crate can capture. Diagnosing why leads to our central contribution: a general security property of build systems that, to our knowledge, no prior work names, every build-time *resolution* decision should be attributable to the unit whose request it satisfies, which we call **resolution integrity**. It comes with a characterization (a resolution channel admits unauthorized cross-unit control only if its inputs are merged, and is exploitable only if that merge is also unscoped) that we validate against Cargo's directive families, corroborate cross-system on CMake, and confirm on an *independent* build-time mechanism (C-preprocessor #include resolution, a pre-registered prediction that held), and it is enforced by *request-level* authorization of each resolution. So the contribution is not that a linker search path can be hijacked (an old, runtime observation) but a named, request-authorized property of build-time resolution, of which the native-link shadow is the consequential witness. This paper makes four contributions.

- (1) **A cross-unit native-link shadow, exploitable in default builds.** Cargo's native-link directives carry authority across compilation units; we demonstrate a weaponized shadow, attacker code running in the shipped binary from an unsigned transitive crate, with the final binary as oracle, including a deterministic OUT_DIR-priority variant, and show it is not limited to a forced configuration: 3 of 7 native-linking crates link system libraries by search-resolved requests in their **default** build, and we capture the crypt32 slot of a real default-features curl (§4, 5).
- (2) **Why per-crate defenses miss it.** The attack survives build-script confinement (shown under Cackle+Bubblewrap, where the shadow still fires) and per-crate review, because the

malice is a *composition* of a per-crate directive with a global, unscoped resolution step, not a code change in any crate (§3, 5).

- (3) **A principled defense.** Provenance-scoped native-link resolution, resolve each request only within its own provenance, is *sound* (machine-checked in Lean 4), *complete w.r.t. declared legitimate use* (shown by executed contrast), and least-restrictive within the family of provenance-based filters (argued, not proved). We realize it three ways (a model, a linker wrapper, an in-tree Cargo patch) and measure strict-mode compatibility on real graphs (§7).
- (4) **Propagation boundaries and ecosystem relevance.** We map which directives cross units (only the native-linking instruction family: `rustc-link-*` and `rustc-flags -L/-l`), ground the shadow against unmodified `libsqlite3-sys 0.38.2`, and measure that 39.5% of 185 resolved graphs already contain a shadowable victim (§5, 8).

We disclosed to the Rust security team, who place it outside their security-response scope (dependency trust) and see no privilege escalation [32], assessments we share: our contribution is the resolution-integrity property and a link-step defense-in-depth mechanism, not a new capability. Executed builds use `cargo 1.98.0`; each section names the artifact that reproduces it.

2 Background: build scripts and the directive channel

A Cargo package may ship a build script (`build.rs`), which Cargo compiles and runs before the package, reading its standard output for directives [27]: a line beginning with `cargo::` (or legacy `cargo:`) is applied to the package’s compilation, every other line ignored. Common directives set a `rustc-cfg` flag, bake a `rustc-env` value, pass a `rustc-link-arg`, or record `rerun-if-changed` inputs. Two features extend who can reach the stream: a build script is ordinary Rust that may call *build-dependency* libraries and spawn processes (running within its process); and a package declaring `links = "name"` may emit metadata that Cargo delivers to a *direct dependent* as `DEP_NAME_K` [27]. Procedural macros are a separate build-time execution surface outside our focus. Our subject is the authority these directives carry, that the native-library directives reach beyond the emitting crate into the final binary’s link (Section 5).

3 Threat model

We consider a developer building a Rust application with Cargo on a trusted host and toolchain. The application’s own crate, its `Cargo.lock`, the Rust toolchain, and Cargo itself are inside the trust boundary. The dependency graph the lockfile resolves is not: any crate in it may be authored or updated by an attacker, through a compromised account, a typosquat, or a malicious release, as in the August 2026 `arrayref` compromise [33]. Concretely, the malicious version enters by the ordinary mechanism that keeps the application itself and its lockfile trusted: a routine cargo update (or the first resolution that admits the release) advances a transitive dependency to the attacker’s version and records it in `Cargo.lock`, which the developer accepts as they accept any dependency update, no change to the application’s own code or to Cargo is needed.

Attacker capabilities. The attacker can publish a crate that enters the victim’s build graph as a dependency of a dependency. A published crate may declare `links`, ship a build script, ship data files, and set the content its build script prints. The attacker cannot modify the victim’s crate, its lockfile pins, the toolchain, or Cargo. We are precise about *which* graph position suffices: the malicious crate must be a *normal* (linked) transitive dependency, because the shadow rides its build script’s `rustc-link-search` into the final binary’s link, and only normal dependencies contribute there. A crate that is *exclusively* a `[build-dependencies]` entry does *not* suffice, we confirm on an executed build that an attacker present only as a build-dependency leaves the victim linking its own library (1111), while the same attacker as a normal dependency shadows it (6666), since

a build-dependency’s link directives apply to host build-script compilation, not to the shipped artifact. This narrows the threat to the demonstrated configuration, a normal transitive dependency with a build script, and is not the loud build-dependency-executes-arbitrary-code threat, which is out of scope below.

What we assume the defender already does. Crucially, we grant the defender the state of the art in build-time review: every crate’s source is available for inspection, and the defender may run source-review tools (cargo-vet, cargo-crev, cargo-audit-build) and capability checkers (Cackle) over the graph. The attack must therefore succeed against a defender who reads and approves the code of every crate.

Goal. The attacker’s goal is to control the *native library* that a *victim* crate’s request resolves to, so attacker-controlled native code links into and runs in the shipped binary, a cross-unit effect on a compilation the attacker is not part of. The distinguishing property, which separates this from prior build-time attacks, is that the effect is **attributed to no crate whose reviewable code the attacker altered**: the victim’s sources are byte-identical, and the shadowing crate emits only an ordinary search-path directive. (Within-build *directive injection*, steering a crate’s own cfg/env, is a secondary facet of the same channel, treated in Section 4 and the supplement; the native-library shadow is the paper’s subject.)

Out of scope. A build script running arbitrary code that directly attacks the host (exfiltration, dropping a payload, tampering with the compiler binary [8, 33]) is the established build-time code-execution threat and is not our subject; it is *loud*, the malicious code is in the crate under review. Our subject is the quieter failure in which the control channel lets one principal act as another. Runtime confinement (sandboxing, network denial) is a complementary defense against the loud threat, tested empirically in Section 5.

Why this matters given code execution already exists. A build script can already run arbitrary code, so the shadow claims no *more* power; its significance is that the effect does not require controlling reviewed code at all, an attacker who changes only a native artifact in an otherwise-unchanged dependency still captures another crate’s request (Section 5.3 demonstrates this on the real winapi-x86_64-pc-windows-gnu crate). So “just edit lib.rs” is no rebuttal: that path is audited and caught, and is unavailable to a binary-only attacker (version-based re-audit is the one partial defense; we do not overclaim past it). The shadow also survives runtime confinement (Section 5, under Cackle+Bubblewrap). We accept Rust’s dependency-trust policy [32]; this is not an attack on it but a gap in the *defense-in-depth* the ecosystem is building, emission-stage confinement (sandboxing, allowlists; Rust’s best-effort goal) is the wrong *stage*, and resolution-stage scoping (Section 7.1) closes it.

4 Cargo’s build-directive channel

Our exploit rides Cargo’s build-directive channel, so we first characterize it. Cargo lets a build script influence compilation by writing directives to its standard output. Each line beginning with cargo: or cargo:: is parsed as a directive (rustc-cfg, rustc-env, rustc-link-arg, metadata, and more) and attributed to the package that declared the script; all other lines are ignored [27]. The channel has three properties that matter for what follows, each a security weakness in its own right: the first two (an unframed, unauthenticated stream) show it places no trust check on directives, and the third, that some directives carry *cross-unit* effect, is this paper’s subject.

In-band. Control and data share one stream: a script’s ordinary output and its directives are the same bytes, marked as a command only by a textual prefix any data can carry.

Unframed. The stream is newline-delimited text; a directive's *value* runs from = to end of line, so a value containing a newline spans into the next directive.

Unauthenticated. Cargo attributes a directive by *position*, not writer: any writer (the script, a spawned process, a linked library) speaks for the crate.

This is the in-band-signaling and confused-deputy vulnerability class (phone phreaking, SQL injection, HTTP request smuggling) in a build system's control plane, orthogonal to per-crate defenses: reviewing or restricting a named crate cannot address an injection that requires no change to its code or is carried by data the victim echoes. The unframed, unauthenticated properties permit *within-build* directive injection (a newline in external data injecting a second directive; a spawned child speaking for the crate), but that is the lesser facet, a build script already runs arbitrary code, so we defer its executed primitives and ecosystem surface to the supplement. This paper's subject is the third property: some directives carry a *cross-unit* effect on a unit the emitter is not part of, which Section 5 develops into the shipped-artifact compromise.

5 Cross-unit native-link authority and the shadow exploit

A build script's directives (Section 4) mostly configure the crate that emits them, and since a build script already runs arbitrary code on the developer's machine, directing its *own* compilation is no escalation of privilege. The sharper question is whether a directive can change a compilation unit its emitter is *not part of*. It can, and the effect reaches the artifact the developer ships. Every attack and propagation witness below is an executed build on cargo 1.98.0 (GNU host toolchain), the cross-unit propagation and native-shadow artifact drivers; the oracle is the final binary's own link outcome or its runtime output, so no self-reported marker is trusted.

5.1 Native directives reach the final binary

Unlike rustc-cfg or rustc-env, the directives rustc-link-lib and rustc-link-search do not act on the emitting crate alone, they propagate to the top-level binary's link. A build script at dependency distance up to 3 *declaring no links key* emits rustc-link-lib=name and the binary's link fails naming *name*: proof the directive crossed into a unit the emitter is not part of. Propagation is conditional on the crate's code being referenced (an unused dependency is dead-code-eliminated), the ordinary case for a live dependency.

An executed distance sweep (the propagation-map driver) maps which directives cross: to distance 3 the native-library family and rustc-flags (which carries -l/-L) reach the final binary, while link-arg, rustc-cfg/env, and DEP_* do not. Section 6 shows this is not a list but the merged-into-the-final-link family, and validates it as a predictive law (Table 1). The attack turns on one asymmetry, links uniqueness gates the DEP_* metadata channel but not rustc-link-search [27], so provider substitution is blocked while search-path pollution is ungated, and the polluting crate carries *no links signal*, so the developer never inspects it for native linking.

5.2 Weaponized shadowing: attacker code runs in the shipped binary

Cross-unit reach becomes a working payload through *library shadowing*. A legitimate -sys-style crate ships a native library libX and emits rustc-link-search=native=dir with rustc-link-lib=X. An unsigned crate at dependency distance 2, with no links key, whose build script emits only a second rustc-link-search pointing at a directory that also contains a libX, causes the final linker to resolve -lX to the *attacker's* library. In our fixture the legitimate symbol returns one constant and the attacker's returns another; the application, which calls the symbol through extern "C", prints the attacker's value, and both libraries are present, so this is genuine search-order shadowing, not the attacker being the sole provider. Attacker native code

is linked into and *executes in the binary the developer ships to end users* (the persistence a build-time-only code-execution model does not capture) from a crate that declares nothing about native linking. The result is deterministic across repeated clean rebuilds.

Which form is vulnerable, precisely. The vulnerable request is the *default* `rustc-link-lib` form (no `static=`), resolved by *searching* the merged `-L` directories, a build-time resolution distinct from runtime loader shadowing (`LD_PRELOAD`) [18]. The `static=` form *resisted* in our tests, and we are precise about why, since it is not inherently an absolute-path reference: `-l static=` still searches the native-library dirs and bundles the archive into the `rlib`. It resisted because the tested `-sys` crates supply their bundled archive from their own `OUT_DIR`, resolved within their own provenance, ahead of an unrelated directory, which is the provenance property, not an absolute-path guarantee. We scope the claim accordingly: the exposure is the search-resolved form drawing on an *external* directory; an attacker placing a directory ahead of a `static=` crate's own output is a case we did not observe and do not rule out. The shadow also carries an ABI burden, it must export the victim's symbols or the link breaks loudly.

Real, unmodified published crates. We ground it against real crates, not only fixtures. An application depends on unmodified `libsqlite3-sys 0.38.2`, whose documented `SQLITE3_LIB_DIR` interface emits the ordinary search-resolved `rustc-link-lib=dylib=sqlite3`: the baseline prints 3045000, and an unrelated distance-2 transitive crate emitting only a search path to its own `libsqlite3` makes the same binary print 6666000 (2 uniform-sample crates request `-lsqlite3` this way, so the interface is ecosystem-real). Run through the published `chainlink-tracker 0.2.0` CLI with a search-resolved victim, the attacker's SQLite constructor writes `attacker sqlite constructor ran` while the command still prints `chainlink 0.2.0`. Here the bundled default is `static=` and resists, a concession narrower than it looks, as the next paragraph shows.

The default build is exploitable, through system libraries. The `static=` form protects a crate's *own* bundled archive, not the *system* libraries it links alongside. Classifying `rustc-link-lib` directives real crates emit in their **default** build (the default-link-classification driver), the vendored library is `static=` and resists, but system libraries are emitted *plain*. Of 7 native-linking crates, 3 do so for a *system* library, `curl-sys`, `libgit2-sys`, `libssh2-sys` request the Windows import libraries `crypt32`, `secur32`, `bcrypt` as ordinary lines. Since any explicit `-L` is searched before the default system directory, an unsigned transitive crate shipping its own `libcrypt32.a` captures the `crypt32` slot: on an executed build the binary runs the attacker's `crypt32` (under both the `plain` and `__imp_/dllimport` conventions). Run end-to-end against a real `curl` at **default** features, the attacker's archive becomes the *sole* `crypt32` provider, the binary retains no `CRYPT32.dll` import (baseline: 16), and attacker code *executes*: a startup constructor fires while the application still runs, no feature flags, no forced configuration (proven with a harmless marker and inert stubs, not a forwarding takeover). The pattern is *cross-platform*, since system libraries are search-resolved by construction. We confirm it executed on Linux: of 11 popular native-linking crates at default features, 7 link a system library by search (`libz-sys`, `openssl-sys`, `libsqlite3-sys`, `curl-sys`, `libgit2-sys`, `libssh2-sys`, `lzma-sys` emit `plain -lz, -lssl/-lcrypto, ...`), and the shadow reproduces across `zlib`, `sqlite3`, and `lzma` (substitution, execution, functionality each confirmed). The attacker carries the ABI burden of the referenced symbols (a partial archive fails loudly).

The attack is engineered, not lucky. Cargo's design says when the attacker's directory is consulted first: build-script search paths accumulate into a topologically-sorted `to_link` vector so a script "will correctly pick up the files they generated" [30], the same ordering that lets an *earlier* participant's directory satisfy a *later* crate's `-l`. Our 4 controlled configurations (the precedence-sweep driver)

confirm precedence is name-determined regardless of declaration order or distance; a direct attacker controls it by naming its crate, a deeply transitive one has weaker control (a stated limit the `OUT_DIR` and system-library routes sidestep).

Removing the precedence precondition. A stronger variant removes the name race: Cargo ranks a crate's own `OUT_DIR` *ahead of every external search path* [30], so an attacker that *generates* the shadow library into its own `OUT_DIR` outranks the provider regardless of order, on an executed build (the `OUT_DIR`-variant driver) an attacker whose name sorts *after* the provider still shadows it (6666) by `OUT_DIR` priority (Section 7: this defeats a defense scoping by path kind, not provenance). The name-race limit thus applies only against a same-rank build-artifact provider; for an external-library provider, control is deterministic.

Why per-crate review does not catch it. The decisive fact is *graph-level*: a reviewer may notice the attacker's shipped library and search directive, but no single-crate review reveals that its path resolves the *victim's* `rustc-link-lib`, a fact about the composition (Section 10). A data-only variant sharpens it, a script emitting a link directive read from its own data file changes the binary across two releases with `build.rs` byte-identical, so source-hashing and API review see nothing. The one defense that catches it is version-based re-audit (cargo-vet, cargo-crev); we claim blindness only against the per-crate source and capability review the build-security discussion centers on.

Under a real sandbox. The community's answer to build-time code execution is to confine the build script (deny network, writes, spawns: the Rust sandbox goal, Cackle's Bubblewrap). We ran the shadow under Cackle+Bubblewrap. Under the realistic policy that authorizes the emission, *the shadow fires under the active sandbox* (`demo_value=6666`): the sandbox governs the *process* and the allowlist the *emission*, but the harm lands when the link step *resolves* -l. On one corpus of the attack and a legitimate native-linking build, the permissive policy that allows the legitimate build's `rustc-link-search` also allows the attacker's, while one restrictive enough to deny the emission also rejects the legitimate build. We do not claim *no* emission-stage policy can ever distinguish the attacker, a targeted audit could reject the specific crate after the fact, but such a policy must know *in advance* which otherwise-permitted search path will conflict with another crate's request (a fact about the composed graph, not the emitting crate), whereas scoping authorizes at the *resolution* step and needs no such foresight. The *prepackaged-library* variant invokes no network/write/spawn API (confirmed), surviving even the strictest confinement; the `OUT_DIR` variant writes and spawns a compiler, firing only under a policy the Rust sandbox goal and Cackle's default allow.

5.3 Binary-only compromise of a real crate

The sharpest form of the threat needs no control over reviewed code. The victim `-sys` crate provider requests `-l demo`; its legitimate provider is *itself* (`links="demo"`, ships its own `libdemo.a`). We add the real, widely-used `winapi-x86_64-pc-windows-gnu`, which ships 1416 opaque prebuilt import libraries and, from a benign build script (our detectors report only benign `env`, `link` capabilities), adds its `lib/` directory to the global native search path, as an *unrelated* dependency: provider has no edge to it and declares it via neither `links` nor `DEP_*`, so it is not in provider's provenance. The attacker's sole capability is planting a colliding `libdemo.a` (6666) into a *vendored* copy of that crate's `lib/`, a path Cargo does not re-verify against the registry checksum (equivalently a pre-packaging or artifact-mirror substitution before checksumming), *not* a rewrite of an already-checksum-verified cache (a stronger capability we do not assume). A checksum taken after substitution covers the malicious binary, so we claim only that *Rust-source* review and source-level tooling are blind, not whole-package checksums or binary-integrity checks.

On an executed real-cargo build (the crate’s build.rs/lib.rs byte-identical to the cached original), stock Cargo satisfies provider’s -l demo from the planted archive; a negative control that keeps the archive on disk but drops the winapi dependency (its search path never reaches the link) links the legitimate library, isolating the cross-crate mechanism; scoped resolution binds the request within provider’s own provenance:

Condition	Rust source	Native archive	Stock Cargo	Scoped
Clean	baseline	legitimate	1111	1111
Binary-only plant	identical	attacker	6666	1111
Plant, winapi off search path	identical	attacker	1111	1111

An attacker need not modify reviewed Rust code to exploit cross-unit resolution: replacing only a native artifact in an otherwise-unchanged, *unrelated* dependency causes stock Cargo to satisfy another crate’s request from the attacker-controlled artifact, while provenance-scoped resolution prevents that transfer of authority. Scoping rejects this *unauthorized* substitution; it does not vouch for the binary of an *authorized* provider (a crate legitimately depending on winapi), which is a binary-integrity concern outside resolution scoping.

Real carrier and real victim, together. This victim is synthetic for a clean oracle; the strongest form, a real, unchanged-source carrier *and* a real victim, runs as one executed build. Planting only libcrypt32.a into the real winapi-x86_64-pc-windows-gnu’s vendored lib/ (binary-only; winapi source byte-identical), against a real *default-features* curl whose curl-sys requests the system library crypt32 by search, stock Cargo builds the real curl binary with the attacker’s crypt32 as the *sole* provider (CRYPT32.dll imports 1→0) and a planted-archive constructor executes in the shipped binary. The capture is *deterministic* (any explicit -L precedes the system directory, so no name race), and scoping restores the system crypt32 (imports back to 1, attacker silent, build intact) since crypt32 is in the requester’s system-default provenance and winapi is not. (A victim supplying its own -L, e.g. libsqlite3-sys, is a retained negative, an unrelated host wins only if it beats that order.)

Ecosystem prevalence. The shadow needs a *shadower* (a no-links search-path emitter) and a *victim* (a search-resolved, non-static= request). Over 472 uniform-sample build scripts (dirscan; lower bounds), 56 (11.9%) emit an unsigaled search path (the lint baseline), but the directly weaponizable crate-shipped shape is only 14 (3.0%) and 57 (12.1%) play victim. At graph scale, of 185 resolved graphs 39.5% are *shadowable-under-attack* (contain a victim; the attacker supplies the shadower, so not exploitable-as-resolved) and 64.4% of those already contain a shadower-shaped crate, a candidate rate, not a proven collision; a bounded entry-surface study (Section 10) sizes how many transitive positions can host one. At the positions themselves substitution is real: placing a harmless substitute archive at a transitive normal-dependency position of 4 real system-linking crates and building stock, the final binary ran the substitute in 3/4 (zlib, sqlite3, lzma), while the bundled-static= crate resisted (the retained negative), reachability at a held position, not a compromise rate.

The cross-crate resolution already happens in the wild. Scanning the same 200 roots for a *confirmed same-name collision* (a crate-shipped emitter shipping lib<X> while a distinct crate requests -l<X>), 2 graphs contain one, over winapi_ntdll: winapi-x86_64-pc-windows-gnu ships libwinapi_ntdll and ntapapi requests -lwinapi_ntdll, the exact cross-unit shape in real graphs, no attacker added. Legitimate in intent (ntapi depends on winapi), it shows the substrate is a live pattern; yet winapi carries *no* links key, so the edge is not a native-provider declaration, under

strict provenance (§7.1, T3) this is undeclared cross-crate provision surfaced as needing a one-line annotation, whose small migration cost §7.2 measures.

6 Build-time resolution integrity: a general property

The native-link shadow is the *witness* for a contribution more general than the exploit: a security property of build-time resolution that, to our knowledge, no prior work articulates. We state it, give a characterization of Cargo’s directive-propagation boundary that we validate against executed evidence, corroborate its cross-system reach on CMake, and give an enforcement we prove sound. What is novel is not that a linker search path can be hijacked, an old, runtime observation, but that a build system forwards *authority over the final artifact* through resolution channels while discarding the *provenance* needed to authorize each request, and that *request-level* authorization of that resolution is the fix.

The property. A build is a graph of compilation *units* that emit directives into shared *resolution steps*; each input carries a *provenance*, the unit that emitted it. A build system has **resolution integrity** when every resolution decision is attributable to the requesting unit’s own provenance, its own paths, the system defaults, and the units it *authorizes* as providers through an explicit declaration (a links/DEP_* edge, not mere dependency reachability and never a filename match). Cargo lacks it: add_native_deps merges every participant’s rustc-link-search into one -L set and resolves every rustc-link-lib against it, so one unit’s directory satisfies another’s request, the principal that declares is not the principal that decides.

Three concepts, kept distinct. The property turns on separating what “crosses units” conflates: (i) an input *crossing a unit boundary*; (ii) an *authorized* input legitimately influencing another unit; and (iii) an *unauthorized* input *controlling a resolution decision* for another unit. Only (iii) is the security failure. DEP_* crosses a boundary (i) but is an authorized, one-hop value a direct dependent voluntarily reads (ii), granting no control over that dependent’s resolution, so it is not (iii). The native-link shadow is (iii): an unauthorized crate’s directory controls another crate’s -l resolution.

A characterization of the directive-propagation boundary. Within this vocabulary the property predicts which Cargo channels admit (iii): only a *merged-consumed* channel (payloads feeding a step assembled from many units) can, and only when that step is also *unscoped*. Merging is thus *necessary but not sufficient*; the operative condition is *unscoped* resolution, which separates this from generic search-path hijacking. We check one prediction per channel and per (merged?, scoped?) condition against executed evidence (Table 1): all 6/6 match. These rows are a complete, boundary-covering account of Cargo’s families, not independent confirmations of a universal law (the native-library and rustc-flags rows exercise the same merged final-link mechanism); generality beyond Cargo rests on the cross-system and independent-mechanism evidence below.

Enforcement, and cross-system reach. Provenance-scoped resolution (Section 7.1) restores the property. We are precise about what is machine-checked versus argued: in Lean 4 we prove *soundness* (scoped_sound: every selected entry is in the requester’s provenance; no_shadow: an out-of-provenance unit is never selected), with an executed contrast that the scoped resolver still returns the legitimate provider. Minimality is *informal*, not a Lean theorem, and only *within the family of provenance-based path filters*: any strictly weaker such filter re-admits a shadow, so provenance-scoping is the least-restrictive sound filter in that family, not a claim over arbitrary policies. The exposure criterion is build-system-level: a system is exposed *iff* it has a merged, unscoped resolution step, which classifies Cargo and directory-scoped CMake link_directories as exposed (both executed) and target-scoped CMake, hermetic Bazel/Nix, and node-gyp as not.

Table 1. Systematic check of Cargo’s directive-propagation boundary against executed evidence. A channel admits unauthorized cross-unit control only if *merged*, and is exposed only if merged *and* unscoped; the merged-&-scoped native-link row shows the operative condition is scoping, not merging. DEP_* crosses one hop but as an authorized value, not resolution control. All 6/6 predictions match.

Channel	Merged?	Scoped?	Authority?	Exposed?	Observed
native-link	yes	no	yes	yes	confirmed
native-link	yes	yes	yes	no	confirmed
rustc-flags	yes	no	yes	yes	confirmed
link-arg	no	n/a	no	no	confirmed
rustc-cfg/env	no	n/a	no	no	confirmed
DEP_*	one-hop	n/a	no	no	confirmed

An independent build-time mechanism confirms the model. The Cargo shadow and the CMake result above both exercise the *linker’s* library search, one mechanism in two systems. To test whether the model predicts a *class* of decisions, we pre-registered a prediction (falsification fixed before observation) for a different resolved decision at compile time: the C preprocessor’s `#include` lookup. From the same criterion: under merged, unscoped `include_directories` an unrelated component’s header directory captures a victim target’s `#include` (predict the attacker’s 6666, not the victim’s 1111), while `target_include_directories(PRIVATE)` confines resolution to the target’s own provenance (predict 1111). All 3 arms matched, each recording its exact compile command and the resolved `cfg.h`. Target-scoped includes are established CMake practice; the contribution here is not discovering that scoping but that our *predicted* exposure boundary (merged-and-unscoped) holds on a second, non-linking resolution mechanism, resolution integrity predicts *and* prevents unauthorized cross-component authority beyond native-library linking.

What request-level authorization adds, precisely. Against the three nearest controls: *linker controls* (`-l:file, static=, rpath`) select which file or kind a request binds, not *by whose provenance* it may be satisfied, so an unrelated directory on the merged `-L` set still answers it; *CMake target scoping* (`target_link_directories PRIVATE`), the closest prior mechanism and credited as such, is a per-target isolation the author opts into at declaration sites, not one *reconstructed and enforced* over a transitively-merged set as Cargo requires (Section 7); and prior Cargo proposals differ in aim, per-crate linking [29] is ergonomic with no authorization semantics, `metabuild` [26] relocates emission and authenticates nothing. None provides *request-level authorization* (each transitive request resolvable only within the requesting unit’s provenance) nor a characterization of which channels need it: that is the contribution, on Cargo, CMake, and an independent mechanism.

7 Defense: authorizing directives and scoping their effects

The exposure has two facets. The *injection* facet, unauthorized directives taking effect within a build (unframed data becoming control, a spawned or linked writer speaking for the crate), is the lesser one, since a build script already runs arbitrary code; it is closed by a different mechanism family (an authenticated, framed channel plus a manifest directive policy Cargo enforces outside the untrusted process), treated in full in the supplement. The rest of this section is the *cross-unit* facet: a legitimate directive from one crate taking effect in a unit it is not part of (the shadow of Section 5.2), which neither writer authentication nor framing touches, the attacker emits its *own* directive legitimately and the harm is in the *resolution*. We take this mechanism further, to a linker wrapper and an in-tree Cargo patch on real builds.

7.1 Scoping the cross-unit effect

The manifest policy governs which directives a build *emits*; it does not stop the shadow of Section 5.2, because there the attacker’s crate emits an ordinary `rustc-link-search` that its own policy legitimately permits. The fault is not an unauthorized emission but an unscoped *effect*: Cargo merges every crate’s search paths into one global set against which it resolves every crate’s `rustc-link-lib`, so one crate’s directory can satisfy another crate’s library. The fix follows from the confused-deputy structure: resolve a crate’s `rustc-link-lib` within only that crate’s own *provenance*, its own search paths, the system default paths, and the search paths of crates it declares as providers (a `links/DEP_*` relationship), to an absolute library path, and link that. A legitimate provider is therefore not inferred from a matching library filename alone: it is either the requesting crate itself, the platform’s system library set, or a crate in the requester’s dependency provenance whose native provider role is declared and reviewable. This keeps `split -sys` patterns working when the dependency relation says who is providing the native object, but it denies a sibling or unrelated transitive crate that merely happens to ship a same-named archive. This is not the `static=` form: it preserves which file is found (a shared library resolves to the shared library), it is per-requesting-crate (two crates requesting the same name can resolve to different files), and it scopes the *search*, not the linkage kind. Cargo assembles the final link line and so can perform this resolution before the single linker invocation.

Three properties make this the enforcement of resolution integrity (Section 6), not merely a patch. **T2 (soundness)**. Under scoped resolution a requester w ’s `-l` resolves only within $\Pi(w)$, which by construction excludes any unit that is neither w , a system path, nor a declared provider of w ; the shadow’s attacker is exactly such a unit, so its path can never be selected, and the shadow’s necessary condition is removed. T2 holds only for a *per-unit* Π : a build-artifact (`OUT_DIR`) path is in $\Pi(w)$ only for the unit that produced it, which is why the `OUT_DIR` variant (Section 5) defeats a scoping by path kind and is blocked only here. **T3 (completeness w.r.t. declared legitimate use)**. Every resolution justified by a *declared* native-provider relationship, provider is w , a system library, or a unit w declares as a provider through `links/DEP_*`, lies in $\Pi(w)$ and is preserved; the only denied case, an *undeclared* cross-crate provider, is surfaced as an annotation rather than silently broken. “Declared” means the explicit `links/DEP_*` relationship Cargo already defines, *not* dependency reachability and not a filename match: a plain dependency edge does *not* authorize native provision. This has a concrete consequence we state plainly, the `winapi/ntapi` pair (`winapi` ships `libwinapi_ntdll` with no `links` key, `ntapi` requests it) is an *undeclared* cross-crate provider even though `ntapi` depends on `winapi`, so strict Π surfaces it as needing a one-line consumer-side declaration, the honest migration cost Section 7.2 measures. (The partial implementations preserve this pair *without* an annotation, since their narrower condition does not fire when no in-scope path competes, a property of the prototypes, not of Π .) **T4 (minimality, within the provenance-based-path-filter family)**. Among mechanisms that filter which search paths a request may use by provenance, the family our resolver, wrapper, and patch belong to, any filter strictly weaker than Π admits, for some requester w , a path from a unit outside $\Pi(w)$ that can ship a library and (search being merged) shadow w ; so Π is the least-restrictive sound filter in this family. We claim this *within* that family, not over arbitrary policies, and separately from the Lean result, which proves soundness, not minimality. We machine-check the core in Lean 4: `scoped_sound` (every entry selected under scoped resolution has in-scope provenance) and `no_shadow` (an out-of-provenance unit is never selected), together with an executed contrast on one configuration, the global resolver returns the attacker (`global_shadowable`) while the scoped resolver returns the legitimate provider (`scoped_keeps_legit`). The development carries no sorry and depends only on `propext`; the replication package ships it (`lake build` is the check). We are precise about *what* is

proved and about the assumptions. The Lean result is a property of the *ideal* resolver over an explicit provenance set Π ; the implementations approximate it and their coverage limits (both wrappers' single interception point; the *permissive best-effort* wrapper's additional permissive fallback, which the *strict* wrapper lacks; and the patch's directory-level pruning that fires only when an in-scope path already supplies the name, so it cannot disambiguate two undeclared same-name providers) are stated in Section 10, not folded into the soundness claim. Minimality likewise assumes Π as defined, a declared provider is a *dependency edge* (links/DEP_*); a stricter native-provider policy would authorize fewer paths and a looser one more, a tradeoff the compatibility upper bound below bounds but does not settle. This scoping is not hypothetical: CMake's `target_link_directories(. . . PRIVATE)` already gives *cross-target* (cross-unit) isolation, and we confirm on executed builds that the shadow fires under CMake's directory-scoped `link_directories` yet is prevented across targets under the target-scoped form (Section 6; the within-target boundary is characterized in Section 10) [12]. The port into Cargo is not mechanical: CMake declares link relationships per target, whereas Cargo forwards directives transitively into one merged `-L` set and applies transitive `-l` from `rlib` metadata at the final link, so the scope has no place to be declared and must be reconstructed from each crate's build-script provenance, which is what our wrapper and in-tree patch do.

We model both policies over real native libraries and execute the link (the resolver-model driver): under the global policy the attacker's library links; under provenance scoping the request resolves to the legitimate library and a self-contained `-sys` crate still links. We also exercise the case the defense could break, a *split -sys* where one crate provides the search path a sibling's `-l` consumes: absent a declared provider the sibling's request fails, which the defense *surfaces* as needing an explicit declaration, and with it resolves. So the defense closes the shadow, preserves the self-contained pattern, and turns illegitimate cross-crate resolution into a declared, reviewable one.

On a real build, two enforcement points. We enforce it on real Cargo builds two ways. First, a linker wrapper installed as Cargo's linker reads Cargo's per-crate build-script output to recover provenance and scopes each request before invoking the real linker; on the cargo 1.98.0 shadow build the stock link takes the attacker's library and the wrapped link the legitimate one. Second, *inside Cargo itself*: a patch to link-line assembly (`add_native_deps` in `src/compiler/mod.rs`) retains the emitting `PackageId` for each search path (which Cargo flattens away) and prunes from the merged `-L` set any *unrelated* directory, from a crate that is neither the requester nor a declared provider, when it would shadow a requested library. Built from source at Cargo commit 694054f34 (self-reported cargo 1.100.0, a development build, 2026-09-23) with `rustc 1.98.0` (commit 88d9e12ae), GNU toolchain, the stock build links `demo_value=6666` and the patched build 1111, including the transitive case where the victim's `-l` is applied from `rlib` metadata at the final link, so pruning at link-line assembly, before `rustc` runs, suffices. The supplement pins the exact commit rather than the version string, since 1.100 is a development version, not a stable release.

The patch preserves legitimate native linking, executed on real crates rather than only argued. Real bundled `-sys` crates build unchanged under it, `zstd-sys`, `libz-sys`, and `bzip2-sys` compile and link with *no* directory pruned. It also leaves the *no-links* sole-supplier shape (`winapi/ntapi`, Section 5) building without an annotation, but we are careful about what that means: it is the *partial prototype* declining to act (its shadow condition fires only when an in-scope path also supplies the name), *not* the policy authorizing the pair. Under the ideal strict Π that same pair is an undeclared cross-crate provider and requires a provider declaration (§7.1, T3); the patch's silence there is a convenience of the directory-level prototype, and we do not present it as enforcement of the policy. Getting the pruning rule right required a refinement that is itself instructive. A first version treated any build-artifact (`OUT_DIR`) path as in scope, reasoning such paths are a crate's own generated

Table 2. One shared adversarial suite, four mechanisms, executed. “own” = the victim’s authorized library links; “shadow” = an attacker’s links; “refuse” = fails closed. The ideal resolver and the strict wrapper are correct in every condition; the permissive best-effort wrapper and the directory-level patch are partial. Alternative-source is a dependency-controlled cargo::rustc-flags=-L.

Adversarial condition	Ideal	Perm. wrapper	Strict wrapper	Patch
authorized provider	own	own	own	own
undeclared provider	own	own	own	own
missing provider	refuse	shadow	refuse	shadow
competing directories	refuse	shadow	refuse	shadow
alternative source (dependency rustc-flags)	own	shadow	own	own

output; but that is exactly the OUT_DIR-priority attack of Section 5, in which an *unrelated* crate generates the shadow library in its own OUT_DIR that Cargo ranks first, and the first version was duly bypassed (demo_value= 6666). The correct invariant is *per-crate* provenance: a build-artifact path is in scope only for the crate that *produced* it (its own OUT_DIR) or a declared links provider, never for an unrelated requester. Under this rule the patch prunes an unrelated directory, external or OUT_DIR, only when an in-scope path already supplies the same name, so a sole no-links supplier is never removed. The refined patch blocks both routes (1111 for the external and OUT_DIR variants alike), while the real bundled -sys crates above still build with nothing pruned. Two honest limits remain: pruning is directory-level (the wrapper is per-file precise); and if a legitimate no-links provider and an unrelated no-links crate ship the same library name, neither is in scope and the in-tree patch does not disambiguate them, that case needs the per-file wrapper or an explicit provider annotation. The supplement ships the patch as a compact diff and the fixtures, so the in-tree enforcement is reproducible.

A shared adversarial matrix: four mechanisms. We exercise *four* mechanisms against one shared suite of 5 adversarial conditions, an *authorized* provider, an *undeclared* provider (the demonstrated shadow), a *missing* provider (no in-scope crate supplies the request), *competing* directories (two undeclared providers of the same name), and an *alternative source*: a directory a *dependency* controls through cargo::rustc-flags=-L (a documented equivalent of rustc-link-search; note a developer’s own RUSTFLAGS/config path is trusted, it is the application’s own provenance, not a dependency’s, and is not this vector). Each cell is an executed oracle (Table 2): the *ideal* resolver (executed model), the *permissive* best-effort wrapper, the *strict* wrapper on real cargo builds (Section 7.2), and the in-tree *patch* (real patched Cargo). The result is unambiguous and is our principal defense result: the ideal resolver and the strict wrapper are correct in *all* 5, linking the authorized library where one exists in scope and failing closed where none does, with no attacker selection. The two partial mechanisms differ in *how* they fall short: the permissive best-effort wrapper gaps on 3 (missing, competing, and the alternative source, whose rustc-flags route its text parse misses), while the directory-level patch, which reads Cargo’s parsed library_paths and so *does* handle the alternative source, still gaps on the 2 conditions with no in-scope supplier to condition its pruning on (missing, competing). So the complete policy *is* realized on real builds by the strict wrapper for the input routes it intercepts (§10), and the two partial mechanisms’ distinct gaps are exactly the ideal-versus-implemented distinction, measured not asserted.

Why not a simpler collision detector? A cheaper defense that fails closed on a same-name clash is a *tradeoff*, not dominance: it *misses* 3 conditions (missing-provider, alternative-source, system-library), each a single-provider substitution with no clash to see, and reverses on only one compatibility

class (a legitimate undeclared provider with a unique basename). Scoping decides by *provenance*, defined for every request, not by a clash that exists only sometimes.

Compatibility. As a regression check, 27/27 real native-linking crates (compression, openssl, curl, rusqlite, git2, zlib variants) build unchanged under the wrapper (0 need a cross-crate declaration); the strict-enforcement result below is the load-bearing measurement. Scoping is not free: a crate that today relies on an *external* search path (the system defaults, or another crate) to resolve a search-resolved library needs an explicit provider annotation only when that external path is another crate. We size the affected set over the uniform sample, token-validated with *dirscan* (the native-link surface artifact driver): of 96 native-linking crates, 73 (76.0%, 95% Wilson 66.6-83.5% within native-linking crates) are unchanged: they provide their own search path, request only *static=* libraries, or emit a search path for others without requesting one themselves. At most 23 (24.0% of native-linking crates, 4.9% of all sampled crates, 95% Wilson 3.3-7.2% of all sampled crates) request a search-resolved library without providing their own search path; these rely on an external path, and we cannot tell per crate whether that path is the system defaults (no annotation) or another crate (annotation needed), so 24.0% is an upper bound on the annotation cost, not a measured breakage rate. An executed check on Linux tightens it: all nine system libraries the default search-resolved crates request (z, ssl, crypto, sqlite3, lzma, ...) are present in system default directories, so they resolve within scope with *no* annotation, and a split provider/consumer fixture fails strict resolution only when the provider is *undeclared* (then links with the declared links/DEP_* edge). Measured strict-mode breakage is thus confined to undeclared cross-crate provision, well below the bound.

7.2 Strict enforcement across real dependency graphs

The 24.0% above is a static upper bound, not a build result. To connect the ideal resolver's guarantee to actual migration cost we *enforce* strict provenance, the ideal policy with no permissive fallback, a cross-crate request that resolves only outside Π fails the build and records the annotation it needs, and build real dependency graphs under it, paired against stock, in a Linux container. We report three arms: 27 real native-linking crates each exercising their native library; 4 application graphs linking *several* system libraries at once (the coexisting-providers case); and a seed-frozen sample of 14 graphs from the graph study's frame (mostly non-native, where strict is a no-op, itself evidence it does not perturb ordinary graphs). Of 45 graphs that build stock, **all 45 build under strict provenance with 0 annotations required** (the six uniform stock failures are unrelated, excluded and retained by name). Zero annotations arise because every native request resolves in the crate's own provenance (self-provided/bundled *-sys*) or a system default directory (system *-sys*), both in Π ; the only shape needing a declaration is undeclared cross-crate provision (winapi/ntapi, §5), absent from this Linux corpus. So strict enforcement builds every real graph we tested unchanged, the migration cost is the rare, surfaced, declared-provider annotation, not broad breakage. One Linux configuration, default features; the annotation count is measured.

The provider declaration, and a migrated build. We specify the declaration exactly and migrate it on a real build. Cargo's links key is *provider-side*, needs a build script, and reaches only direct dependents, so it is not the consumer-side authorization strict scoping needs. We propose a consumer-side manifest field, written by the crate whose *-l* request it is (or the application), naming per library the provider crate it authorizes:

```
[native-provenance]
foo = "provider" # authorize crate 'provider' to supply -lfoo
```

The strict linker honors it: a cross-crate request resolvable only via another crate's directory is refused *unless* the consumer authorized that provider, then resolves within exactly that provider's directory. We migrate the *actual* winapi/ntapi relationship end-to-end on the real x86_64-pc-windows-gnu target: ntap emits rustc-link-lib for 5 winapi import libraries, all supplied by the separate no-links winapi-x86_64-pc-windows-gnu crate's search path. Under strict provenance with *no* declaration the build **fails closed** on all 5; adding 5 one-line declarations naming that one provider crate, it **builds**, a measured annotation cost of 5 lines to a single crate. The field is a proposed Cargo addition our linker enforces. Its semantics are per-requester: an entry in crate *R*'s manifest authorizes *R*'s request for the named library, keyed by (requesting crate, library name), not by name globally, so a sibling must declare its own provider (keeping the confused-deputy guarantee). The provider is a dependency resolved in *R*'s own subtree, inheriting Cargo's version and target resolution, distinct versions are distinct packages with distinct provenance, and a target-specific provider authorizes only where that dependency resolves. When two dependencies supply the same basename, each requester names the one it intends, the disambiguation the directory-level patch cannot do (T3).

The strict wrapper enforces the complete policy for the routes it intercepts; the in-tree patch is partial. The *strict linker wrapper*, installed as Cargo's linker, enforces the ideal fail-closed policy end-to-end over the input routes it intercepts: across all five adversarial conditions as real cargo builds (victim $\neq$ attacker) it passes 5/5 with *no* attacker selection, linking the authorized/own library where one exists in scope (including alternative-source, scoping a dependency-controlled rustc-flags=-L by its emitter's provenance) and failing closed on the 2 with no authorized provider (missing, competing). It does not intercept every search-path route (§10), which an in-tree implementation would; we do not claim completeness across every Cargo configuration. The *in-tree Cargo patch* is a *partial* directory-level variant: it blocks the demonstrated shadow and preserves the authorized build but still admits the attacker in the 2 harder conditions (Table 2), since directory pruning cannot do per-request resolution.

What full in-tree enforcement requires. A production fix belongs in Cargo's link-line assembly (add_native_deps; diff-level sketch in the artifact): *retain provenance* (keep each search path's emitting PackageId, discarded today when the paths flatten into one -L set) and *move resolution* per-crate. The subtle part: a transitive crate's requests ride in rlib metadata resolved by rustc against the merged set, so enforcement must resolve each within the requester's provenance before the link, keying on each path's *origin* to cover rustc-link-search, rustc-flags, and config paths uniformly, what the wrapper does at the link it intercepts, applied everywhere. This is why existing defenses miss the shadow by *stage*: version audit acts at *introduction*, and sandboxing, capability allowlists, and directive policy at *emission*, but the shadow is decided at *resolution* (links uniqueness acts there but gates only the metadata channel, not search paths). Scoped resolution bounds *where* an emitted directive takes effect without judging what a correctly scoped one then does, control-channel integrity, not malware detection, so it composes with runtime confinement.

8 An over-privileged, helper-written channel

The shadow is dangerous because of its setting: build-time authority is vast relative to how it is used, and the code that wields it is usually not the code a reviewer reads. In a uniform sample of 5908 crates.io crates, 66.5-78.2% of build scripts declare no artifact-producing dependency and 59% emit nothing that changes compilation, the leverage is in the directives, not the code. And the directive is often authored outside the declaring package: across 121 frozen captures of 14 applications, all 1936 third-party build-script instances link a third-party helper (exposure to unread build code), and, an authorship-level lower bound, since linkage alone does not establish emission,

160/1936 (8.3%) link a helper that *itself emits* a compilation-affecting directive from its library sources (supplement: full helper-emission counts).

9 Related work

Build-time code execution and its defenses. That a Cargo build script runs arbitrary code is well established as an attack vector [14, 33]; Counterexamples in Safe Rust shows a `build.rs` tampering with the compiler [8], the build-time echo of Thompson’s trust [34]. The proposed defenses share an object, the code or capabilities of a *named crate*: Cargo Vet and cargo-crev audit versions and diffs [2, 19]; cargo-audit-build stores trusted `build.rs` content [1]; Capslock infers permitted effects [6]; Cargo Scan follows effects across crates and lists build-time analysis as future work [38]; Cackle and cargo-sandbox sandbox scripts [15, 17]; Pappas and Gazzillo check build phases against declared permissions [24]; Rust discussions weigh allowlisting and Wasm [25, 31]; and MALOSS, DONAPI, Latch, and Ferreira et al. detect malicious packages or gate on permission change [4, 5, 10, 37]. Each reasons about *which crate* may act, or confines its build script. Our attack is orthogonal to both: the shadower’s reviewed code is unremarkable and it survives confinement because emitting a directive is all it does (Section 5); the one exception we do not overclaim past is version-based re-audit.

Library and search-path hijacking. Redirecting a program to a malicious library by search order is a known *runtime* attack (LD_PRELOAD, LD_LIBRARY_PATH, RPATH [18], with a loader-level defense binding each object to a cryptographic identity [23]). Those operate in the deployed loader; we show the same confusion one layer earlier and in a different actor, Cargo’s build-time, cross-crate link resolution, where a transitive dependency plants the shadow into the shipped artifact and the defenses in play are per-crate review and confinement, not the loader. It is distinct from dependency confusion (which substitutes a package by *name* at resolution time): our shadow substitutes a *native library* at link time within a legitimately resolved graph. The community has requested per-crate native linking for *ergonomic* reasons [29]; we give the security rationale that request does not address.

In-band signaling and the confused deputy. The class we identify is old: a control channel that shares its medium with data and authenticates no writer, underlying in-band phone tones, SQL and command injection, HTTP request smuggling, and the confused-deputy problem [7]. The contribution is not the class but its unrecognized instance, a mainstream build system routing compilation decisions through an unframed stream whose directives it does not authorize, and that the standard remedy (framed delivery with message authorization) applies. To our knowledge no prior work models Cargo’s directive stream, the DEP_* metadata path, or spawned-process output as a security boundary; the Cargo reference [27] documents the mechanism without a trust statement. RFC 2196 (metabuild) [26] relocates `build.rs` into a named build-dependency crate for maintainability, but that crate still emits directives on behalf of a package that merely names it, the cross-principal arrangement our primitive exploits, so it authenticates nothing.

Provenance, attestation, and transparency. Supply-chain provenance and signing frameworks (SLSA, in-toto, Sigstore) and software bills of materials (SPDX, CycloneDX) certify *how* an artifact was built and *what* it contains [16, 20, 21, 22, 35]. They are orthogonal to the shadow: the artifact is faithfully built from the resolved inputs and its inventory is unchanged; what none records is which native library file a crate’s `-l` resolves to *within* the build, exactly where the shadow acts. Closer still, cloud and hermetic build systems *record* resolution provenance or require whole-input declaration. Our property is adjacent but distinct: neither recording nor whole-input declaration,

but *request-level authorization*, each transitive native-library request is resolvable only within the requesting unit's own provenance, so a recorded-but-unauthorized input cannot satisfy it.

Measuring the ecosystem. Empirical supply-chain studies map dependency graphs and vulnerability propagation [9, 11, 13] and update scoping [3]; LastPyMile and in-toto protect the source-to-artifact and pipeline-integrity transitions [35, 36]. Our measurements serve a narrower end, sizing who can write the directive channel, over a uniform random sample of published build-script source.

10 Threats to validity

Not a single-version accident; propagation coverage. The shadow is executed on cargo 1.98.0 (GNU host) but rests on long-standing documented `-sys` forwarding [27] and fires unchanged on 3 stable releases (1.74.0-1.98.0). The propagation map exercises a top-level *binary* target; the claim we rely on, the search-resolved native-library family reaches the final binary while `link-arg`, `rustc-cfg`, and transitive `DEP_*` do not, is established for that target, consistent with the Cargo reference and issue #9554 [28], and argued exhaustive: the map enumerates every `cargo::` family the reference defines and only the native-library family crosses. A family outside that set, or added by a future Cargo, is out of scope.

Precedence and ABI realism. The search-order result rests on 4 controlled configurations pinning a name-determined, attacker-influenceable precedence (order and distance have no effect; the losing case confirms necessity). We do not derive Cargo's exact traversal, and a deep attacker not naming its subtree's entry crate has weaker control over the name-ordered route, but that applies only to the name-race variant; the `OUT_DIR` variant outranks external paths by Cargo's kind ordering and needs no favorable name. Separately, a stealthy shadow must reimplement the exact symbols the victim uses or the link fails loudly; our witness reimplements the single symbol the application calls, claiming no drop-in replacement for an arbitrary library.

Prevalence is a token-validated lower bound; co-occurrence is not exploitability. The role counts use the token detector `dirscan`, so they exclude directives in comments or strings but miss run-time assembled ones, lower bounds; we report the crate-shipped shadower shape (3.0%), not the broader search-emitter rate. At graph scale, 73/185 resolved graphs are shadowable-under-attack and 47/73 exhibit a *natural role co-occurrence* (a crate-shipped search emitter and a victim together); the 15 unresolved roots do not change this (36.5-44.0% of all roots). A co-occurrence is not a proven exploitable collision, the ingredients coexist, but need not meet over one name with attacker-favourable order, so we make no exploitable-graph rate claim.

The entry surface counts positions, not verified propagation. The 36-graph attack-position study (seeded sample, resolved with cargo metadata) counts only transitive build-script packages reachable along *normal* (non-build, non-dev) edges; build-only packages, which Section 3 shows cannot shadow, are excluded (101 removed, median 12→9.5). A position's presence does not establish its directive reaches a specific victim's final link: it bounds how *many* positions exist, not that any is exploitable as resolved.

The defense is evaluated from a Cargo source tree, not upstreamed. We evaluate it three ways (a model against real native libraries; a linker wrapper on a real build; an in-tree patch to `add_native_deps`), build and run the patch but do not submit it upstream or modify `rustc`. The static 24.0% is an upper bound on annotation cost; the measured strict-build result (§7.2) supercedes it. The wrapper intercepts the single linker invocation where every `-l` resolves and covers the routes we test, including the alternative-source condition of Table 2 (a *dependency-controlled*

cargo: rustc-flags=-L Cargo forwards there). It does not read paths injected by other routes, *application-level* RUSTFLAGS (set by the developer, not a dependency, hence trusted), linker scripts, indirect invocations, a limit of the interception point, not the invariant, since scoping is defined over each path’s *origin* and an in-tree implementation covers those routes. We thus claim the strict wrapper completely enforces the policy for the tested and intercepted routes, not unrestricted completeness across every Cargo configuration.

Why per-crate review and confinement miss it. We claim no *single-crate* review surfaces the shadow, not that the shadower is individually innocuous: a reviewer may note a no-links crate shipping a library and a search path, but one crate cannot reveal whether the shipped name collides with another’s request under an attacker-favourable order, a fact about the composed graph, further hidden by the OUT_DIR variant. Confinement misses it categorically: the shadow fires under Cackle+Bubblewrap, and a policy strict enough to flag the emission would deny an instruction legitimate native-linking crates need, since the sandbox governs the build-script process while the shadow resolves at the later link step.

Sampling and its two non-response counts. All ecosystem samples are frozen (with recorded hash, seed, toolchain, and raw failure set) before the classifiers or graph predicates run, from one crates.io snapshot. The frame is crate-level, not download-weighted, so the uniform build-script sample estimates crate-level prevalence on one host toolchain (92 unresolved crates retained, no owner-deduplication); we do not extrapolate to a usage-weighted population. The graph study is a *separate* seeded sample (15/200 failed, retained) reporting the resolved denominator and worst-case bounds; the wrapper corpus is a frozen compatibility smoke test.

Generality beyond Cargo, precisely. We confirm the shape with executed CMake builds [12] on two decisions, directory-scoped link_directories and include_directories each let one target’s directory capture another’s request, while the target-scoped forms prevent the cross-target leak (Section 6). Only the Cargo and CMake cases are executed; classifying node-gyp, Nix, and Bazel as unexposed is an *architectural analysis* of documented isolating designs, *conditional on those properties being enforced*.

Default-build exposure: scope and platforms. The default-build result is executed on two hosts (Windows/MinGW; Linux, 7/11 search-resolved). The crate sets are *curated popular* crates, so that fraction is a rate over the named set, not a population estimate; and the real-curl witness proves code execution and full displacement with inert stubs.

11 Conclusion

Cargo’s native-link directives carry authority across compilation units: an unsigned transitive crate’s rustc-link-search takes effect in the top-level binary’s link, running attacker code in the shipped artifact, including, because system dependencies are search-resolved by default, in the *default* build of the HTTP/git/SSH stacks. It survives build-script confinement and per-crate review because the malice is the *composition* of a per-crate directive with a global, unscoped resolution step, not code in any one crate, indeed, an attacker who changes only a vendored binary, not reviewed source, still captures another crate’s link. The fix follows from the diagnosis: *provenance-scoped* resolution binds each request to its own provenance, preserving the legitimate -sys patterns (all 45/45 tested graphs build at zero annotations); it is machine-checked sound in Lean 4 and enforced end-to-end by the strict linker wrapper. The Rust team places this outside their security-response scope (dependency trust) and sees no privilege escalation, which we accept; the point is not a new capability but that scoping this authority is a precondition for any build-time supply-chain defense-in-depth that reasons about which crate did what.

Disclosure and ethics. Disclosed to the Rust Security Response WG before any public deposit (no CVE), to be filed publicly after review; harmless local fixtures only, no credentials or human data.

Data Availability

The replication package contains the cross-unit and shadow drivers, the propagation map, the real-library witness, the graph study, the token-based directive detector with its regression suite and frozen corpus, the modelled/wrapper/in-tree defense prototypes, the Cackle/Bubblewrap reproduction, the real native-linking compatibility corpus, the cross-system CMake and independent-mechanism (C-preprocessor `#include`) experiments with their pre-registered predictions, the limited-trust build-input experiments (the vendored-binary tamper and the real-carrier/real-victim curl witness), the pinned Docker images, and the Lean 4 development that machine-checks soundness (lake build; no sorry, only propeXt). A single document lists the command and pinned toolchain for each number; analysis scripts refuse to emit a table whose evidence changed, and every sampling frame is frozen before outcomes. The resolution path is reviewer-reproducible through *harmless* witnesses (controlled linker tests and end-to-end builds displacing a system library with inert stubs plus a startup marker, on Windows and Linux); only the strongest weaponized form is withheld during review. The in-tree Cargo patch is real and executed (a diff with fixtures, built from a Cargo source tree), distinct from the non-executed upstream-integration sketch.

References

- [1] cargo-audit-build contributors. 2026. cargo-audit-build: Audit Cargo build.rs Files. <https://docs.rs/crate/cargo-audit-build/0.3.2/source/README.md>. Version 0.3.2 documentation; accessed 2026-09-14.
- [2] cargo-crev contributors. 2026. cargo-crev: A Cryptographically Verifiable Code Review System for Cargo. https://docs.rs/cargo-crev/latest/cargo_crev/doc/user/getting_started/index.html. Documentation includes crate-version verification and version-diff review; accessed 2026-09-14.
- [3] Yaniv David, Xudong Sun, Raphael J. Sofaer, Aditya Senthilnathan, Junfeng Yang, Zhiqiang Zuo, Guoqing Harry Xu, Jason Nieh, and Ronghui Gu. 2022. UPGRADVISOR: Early Adopting Dependency Updates Using Hybrid Program Analysis and Hardware Tracing. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, USA, 751 to 767.
- [4] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, Reston, VA, USA, 17 pages. doi:10.14722/ndss.2021.23055
- [5] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2021. Containing Malicious Package Updates in npm with a Lightweight Permission System. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1334 to 1346. doi:10.1109/ICSE43902.2021.00121
- [6] Google. 2026. Capslock: Capability Analysis for Go. <https://github.com/google/capslock>. Accessed 2026-09-13.
- [7] Norm Hardy. 1988. The Confused Deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36 to 38. doi:10.1145/54289.871709
- [8] Muhammad Hassnain and Caleb Stanford. 2024. Counterexamples in Safe Rust. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. Association for Computing Machinery, New York, NY, USA, 128 to 135. doi:10.1145/3691621.3694943
- [9] Joseph Hejderup, Moritz Beller, Konstantinos Triantafyllou, and Georgios Gousios. 2022. Präzi: From Package-Based to Call-Based Dependency Networks. *Empirical Software Engineering* 27, 5 (2022), 102. doi:10.1007/s10664-021-10071-9
- [10] Cheng Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. DONAPI: Malicious NPM Packages Detector Using Behavior Sequence Knowledge Mapping. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, USA, 3765 to 3782.
- [11] Peiyang Jia, Chengwei Liu, Hongyu Sun, Chengyi Sun, Mianxue Gu, Yang Liu, and Yuqing Zhang. 2022. Cargo Ecosystem Dependency-Vulnerability Knowledge Graph Construction and Vulnerability Propagation Study. arXiv:2210.07482. doi:10.48550/arXiv.2210.07482
- [12] Kitware. 2024. CMake Documentation: target_link_directories and link_directories. https://cmake.org/cmake/help/latest/command/target_link_directories.html. Per-target link-directory scoping; the link_directories page recommends absolute paths to avoid linking the wrong library.
- [13] Raula Gaikovina Kula, Katsuro Inoue, and Christoph Treude. 2023. Promises and Perils of Mining Software Package Ecosystem Data. In *Software Ecosystems: Tooling and Analytics*. Springer International Publishing, Cham, 63 to 81. doi:10.1007/978-3-031-36060-2_3

- [14] Piergiorgio Ladisa, Merve Sahin, Serena Elisa Ponta, Marco Rosa, Matias Martinez, and Olivier Barais. 2023. The Hitchhiker’s Guide to Malicious Third-Party Dependencies. In *Proceedings of the 2023 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. Association for Computing Machinery, New York, NY, USA, 65 to 74. doi:10.1145/3605770.3625212
- [15] David Lattimore and Cackle contributors. 2023. Cackle: Code ACL Checking for Rust. <https://github.com/cackle-rs/cackle/blob/main/CONFIG.md>. Accessed 2026-09-13.
- [16] Linux Foundation. 2021. SPDX: System Package Data Exchange (ISO/IEC 5962:2021). <https://spdx.dev>.
- [17] Martin Madsen. 2026. cargo-sandbox: Sandboxing Cargo Compiler and Build-Script Invocations. <https://github.com/madsmtn/cargo-sandbox>. Accessed 2026-09-14.
- [18] MITRE ATT&CK. 2024. Hijack Execution Flow: Dynamic Linker Hijacking (T1574.006). MITRE ATT&CK technique T1574.006. Runtime dynamic-loader library hijacking via LD_PRELOAD and search paths.
- [19] Mozilla and Cargo Vet contributors. 2026. cargo-vet: Supply-chain security for Rust. <https://mozilla.github.io/cargo-vet/specifying-policies.html>.
- [20] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. 2022. Sigstore: Software Signing for Everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM. doi:10.1145/3548606.3560596
- [21] Open Source Security Foundation. 2023. SLSA: Supply-chain Levels for Software Artifacts. <https://slsa.dev>.
- [22] OWASP Foundation. 2023. CycloneDX Software Bill of Materials Standard. <https://cyclonedx.org>.
- [23] Can Ozkan and Dave Singelee. 2026. Resolving the Correct Library: A Loader-Level Defense Solution Against Shared Object Hijacking. arXiv preprint arXiv:2605.26665. <https://arxiv.org/abs/2605.26665> Runtime dynamic-loader defense; not build-time or package-manager resolution.
- [24] Brent Pappas and Paul Gazzillo. 2026. Build Code is Still Code: Finding the Antidote for Pipeline Poisoning. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE. arXiv:2601.08995.
- [25] Rust Internals contributors. 2021. Sandbox build.rs and proc macros. <https://internals.rust-lang.org/t/sandbox-build-rs-and-proc-macros/16345>.
- [26] Rust RFC 2196 authors. 2017. RFC 2196: metabuild. <https://rust-lang.github.io/rfcs/2196-metabuild.html>.
- [27] The Cargo Book. 2026. Build Scripts. <https://doc.rust-lang.org/cargo/reference/build-scripts.html>.
- [28] The Cargo Project. 2021. rustc-link-arg does not propagate transitively. rust-lang/cargo issue #9554.
- [29] The Cargo Project. 2023. Per-crate linking option in Cargo manifest. rust-lang/cargo issue #12848.
- [30] The Cargo Project. 2026. BuildScripts (to_link): topologically ordered build-script link paths. cargo core::compiler::BuildScripts source documentation.
- [31] The Rust Project. 2024. Explore Sandboxed Build Scripts. <https://rust-lang.github.io/rust-project-goals/2024h2/sandboxed-build-script.html>.
- [32] The Rust Project. 2024. Security Policy. <https://www.rust-lang.org/policies/security>.
- [33] The Rust Security Response WG. 2026. Supply chain attack on arrayref. <https://blog.rust-lang.org/2026/08/20/supply-chain-attack-on-arrayref/>.
- [34] Ken Thompson. 1984. Reflections on Trusting Trust. *Commun. ACM* 27, 8 (1984), 761 to 763. doi:10.1145/358198.358210
- [35] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Capps. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, USA, 1393 to 1410.
- [36] Duc-Ly Vu, Fabio Massacci, Ivan Pashchenko, Henrik Plate, and Antonino Sabetta. 2021. LastPyMile: Identifying the Discrepancy Between Sources and Packages. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 780 to 792. doi:10.1145/3468264.3468592
- [37] Elizabeth Wyss, Alexander Wittman, Drew Davidson, and Lorenzo De Carli. 2022. Wolf at the Door: Preventing Install-Time Attacks in npm with Latch. In *Proceedings of the 2022 ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 1139 to 1153. doi:10.1145/3488932.3523262
- [38] Lydia Zoghbi, David Thien, Ranjit Jhala, Deian Stefan, and Caleb Stanford. 2026. Auditing Rust Crates Effectively. In *Programming Languages and Systems: ESOP 2026 (Lecture Notes in Computer Science, Vol. 16502)*. Springer, Cham, 424 to 444. doi:10.1007/978-3-032-22723-2_15